

Videogame Semantic Search

Relazione Tecnica di Progetto

Ontologia OWL 2 per videogiochi: modellazione, reasoning e ricerca semantica

Videogame Semantic Search Project

5 agosto 2026

Versione 1.0

*Documento tecnico-accademico prodotto nell'ambito del progetto
di Semantic Web — Ontologia e Knowledge Graph per Videogiochi*

Sommario

Il presente documento descrive il progetto **Videogame Semantic Search**, un sistema di ricerca semantica che integra un'ontologia OWL 2 modellata sul dominio dei videogiochi, un knowledge graph popolato da Wikidata e un'interfaccia web con grafo interattivo. L'ontologia, costruita secondo i principi del Semantic Web, definisce 78 classi, 31 object property, 26 datatype property e 17 classi definite con reasoning automatico. La popolazione da Wikidata (2010–2026) produce oltre 1.1 milioni di triple, interrogabili in linguaggio naturale tramite un agente SPARQL basato su GPT-4.1-mini. Vengono approfonditi in particolare gli aspetti di modellazione dello schema dell'ontologia, le scelte di design ontologico, il sistema di reasoning a due livelli (OWL 2 RL + CONSTRUCT) e le tecniche di validazione adottate.

Indice

Indice	2
1 Introduzione	5
1.1 Contesto: Semantic Web e Ontologie	5
1.2 Obiettivo del Progetto	5
1.3 Casi d'Uso	5
1.4 Panoramica della Relazione	5
2 Architettura del Sistema	6
2.1 Diagramma Architetture	6
2.2 Stack Tecnologico	6
2.3 Flusso Dati	6
2.4 Cache a Due Livelli	7
2.5 Tabella dei Componenti	7
2.6 API Endpoint	7
3 Panoramica dell'Ontologia	8
3.1 Dominio	8
3.2 Namespace e Separazione Schema/Istanze	8
3.3 Fonte Dati: Wikidata	8
3.4 Pipeline di Popolamento	8
3.5 Statistiche Riassuntive	9
4 Classi dello Schema dell'Ontologia	9
4.1 Gerarchia delle Classi	9
4.2 Tabella Completa delle Classi Primitive	9
4.3 Scelte di Modellazione	11
4.3.1 Partizione di Platform	11
4.3.2 Generi come Classi Aperte	12
4.3.3 GameMode come Classe Top-Level Separata	12
4.3.4 Review e GameEvent: Pattern N-ary Relation e Lifecycle Event	12
5 Proprietà dello Schema dell'Ontologia	13
5.1 Object Property	13
5.2 Datatype Property	13
5.3 Property Chain Axiom	14
5.4 Proprietà Transitive	14
5.5 Coppie Inverse Sistematiche	15
6 Classi Definite	15
6.1 Panoramica	15
6.2 Analisi per Categoria	16
6.2.1 Mode-derived	16
6.2.2 Platform-derived	16
6.2.3 Sequel-derived	17
6.2.4 Quality-derived	17
6.2.5 Award e Franchise (preesistenti)	17
6.2.6 Cardinality (solo necessarie)	17
6.3 Costrutti OWL 2 Utilizzati	17

7	Disjointness	18
7.1	Blocchi AllDisjointClasses	18
7.2	Importanza delle Disjointness	18
7.3	Nota su SinglePlayerGame e MultiplayerGame	18
8	Allineamento Esterno	19
8.1	equivalentClass verso Wikidata	19
8.2	equivalentProperty verso Dublin Core	19
8.3	Vantaggi dell'Allineamento	19
8.4	Limitazione Attuale	19
9	Reasoning	20
9.1	Architettura a Due Livelli	20
9.1.1	Livello 1: owlrl (OWL 2 RL)	20
9.1.2	Livello 2: construct_reasoner.py (SPARQL CONSTRUCT)	20
9.2	Perché Alcuni Costrutti Non Sono in OWL 2 RL	21
9.3	Bug Critico: owl:sameAs Falsi e Soluzione	21
9.3.1	Il Problema	21
9.3.2	La Soluzione	21
9.3.3	Impatto	22
9.4	Materialiser Mirato in enrich_owl.py	22
10	Popolamento dell'Ontologia	22
10.1	Pipeline populate_wikidata.py	22
10.1.1	Query SPARQL verso Wikidata	22
10.1.2	Deduplicazione	23
10.1.3	OWL-RL Reasoning	23
10.1.4	Pruning	23
10.2	Demo Subset 2020–2026	23
10.3	Performance: rdflib vs pyoxigraph	23
11	Validazione dello Schema	24
11.1	validate_tbox.py	24
11.2	Risultato della Validazione	24
11.3	Analisi dei 18 Warning	24
11.3.1	12 Warning INVERSE_NOT_BIDIRECTIONAL	24
11.3.2	6 Warning MISSING_DOMAIN / MISSING_RANGE	24
12	Frontend e Backend	25
12.1	Frontend: React + TypeScript + Vite	25
12.1.1	Componenti Principali	25
12.1.2	Caratteristiche del Grafo Interattivo	25
12.2	Backend: FastAPI Async	25
12.2.1	SPARQL Agent	26
12.2.2	GraphBuilder	26
12.2.3	Cache a Due Livelli	26
12.3	Endpoint API	26
13	Conclusione	26
13.1	Riepilogo dei Risultati	26
13.2	Possibili Estensioni Future	27

A Esempio di Istanze di Test	27
A.1 Risultati della Verifica	28
B Sintassi DL delle Classi Definite	28
Bibliografia	28

1 Introduzione

1.1 Contesto: Semantic Web e Ontologie

Il Semantic Web, introdotto da Tim Berners-Lee nel 2001, rappresenta un'estensione del Web tradizionale in cui l'informazione è strutturata e dotata di semantica formale, consentendo alle macchine di interpretare e ragionare sui dati in modo automatico. Il fondamento teorico di questa visione risiede nel concetto di **ontologia**, definita da Gruber [1993] come <una specifica esplicita e formale di una concettualizzazione condivisa>.

Le ontologie, espresse in linguaggi formali come OWL 2 [Hitzler et al., 2012], permettono di modellare un dominio di conoscenza attraverso:

- **Classi** — categorie di entità (es. `VideoGame`, `Developer`);
- **Proprietà** — relazioni tra entità (es. `developedBy`) e attributi (es. `metacriticScore`);
- **Assiomi** — vincoli logici che arricchiscono la semantica (es. classi definite, disjointness, catene di proprietà).

Un knowledge graph è la concretizzazione pratica di un'ontologia popolata con dati reali: nodi (istanze) e archi (asserzioni RDF) formano un grafo interrogabile tramite query SPARQL.

1.2 Obiettivo del Progetto

Il progetto **Videogame Semantic Search** si propone di:

1. **Costruire un'ontologia OWL 2 di alta qualità** sul dominio dei videogiochi, coprendo il periodo 2010–2026, con uno schema ricco di assiomi (classi primitive e definite, disjointness, property chain, allineamenti esterni);
2. **Popolare l'ontologia da Wikidata** tramite query SPARQL federate, producendo un insieme di istanze di oltre 1.1 milioni di triple;
3. **Abilitare l'interrogazione in linguaggio naturale** attraverso un agente SPARQL basato su Large Language Model (GPT-4.1-mini) che traduce domande in italiano in query SPARQL eseguibili;
4. **Visualizzare i risultati come grafo di conoscenza interattivo**, navigabile con zoom, pan, filtri e dettaglio nodi.

1.3 Casi d'Uso

I principali casi d'uso del sistema sono:

- **Ricerca semantica** — interrogare il knowledge graph con domande come <Quali giochi ha sviluppato FromSoftware?> o <Top 10 giochi RPG con Metacritic più alto>;
- **Grafo di conoscenza interattivo** — esplorare visualmente le relazioni tra giochi, sviluppatori, generi, piattaforme e franchise;
- **Reasoning automatico** — classificare automaticamente i giochi in categorie derivate (MultiplayerGame, HighlyRatedGame, SequelGame, ecc.) sfruttando gli assiomi OWL 2.

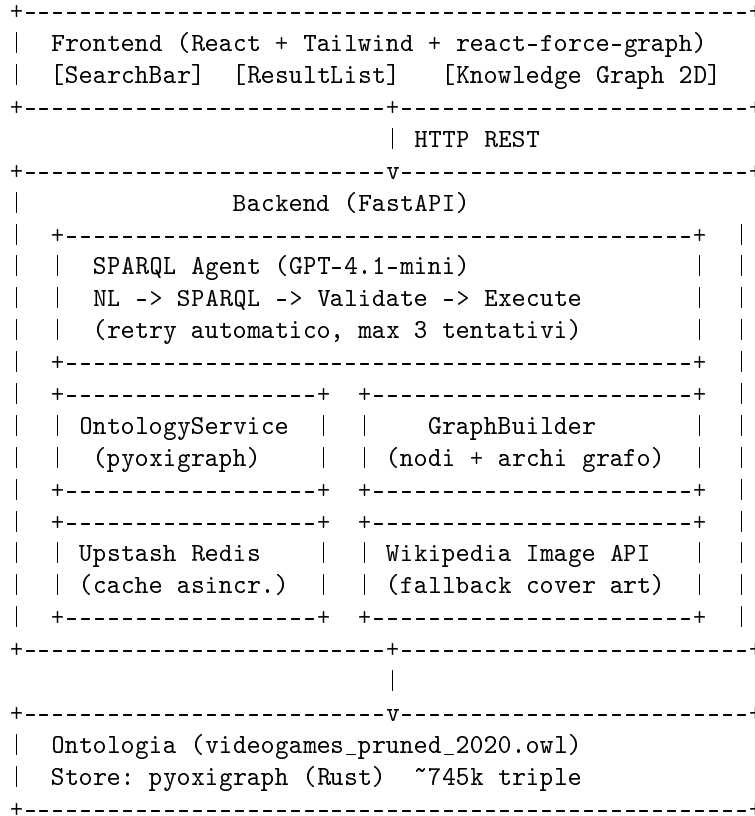
1.4 Panoramica della Relazione

Il presente documento è organizzato come segue. La Sezione 2 descrive l'architettura software complessiva. Le Sezioni 3–8 costituiscono il cuore della relazione e analizzano in dettaglio l'ontologia: lo schema dell'ontologia (classi, proprietà, classi definite, disjointness) e allineamento esterno. La Sezione 9 approfondisce il sistema di reasoning a due livelli, incluso il bug critico degli `owl:sameAs` falsi e la sua risoluzione. La Sezione 10 descrive la pipeline di popolamento da Wikidata. La Sezione 11 illustra i controlli di validazione dello schema. La Sezione 12 presenta l'applicazione web. La Sezione 13 riassume i risultati e delinea possibili sviluppi futuri.

2 Architettura del Sistema

2.1 Diagramma Architeturale

L'architettura del sistema segue un classico pattern a tre livelli (three-tier): frontend di presentazione, backend API REST e store ontologico. Il diagramma seguente illustra il flusso dati:



2.2 Stack Tecnologico

La Tabella 1 riassume le tecnologie impiegate per ciascun livello.

Tabella 1: Stack tecnologico del sistema

Layer	Tecnologia
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Grafo interattivo	react-force-graph-2d
Backend	FastAPI (Python 3.12+, async)
Ontologia	pyoxigraph (Rust), OWL 2, owlrl (rdflib)
LLM / Agente	OpenAI GPT-4.1-mini
Cache	Upstash Redis (async) + cache in-memory
Fonte dati	Wikidata SPARQL
Deploy	Render (backend) + Vercel (frontend)

2.3 Flusso Dati

Il flusso di una tipica interrogazione è il seguente:

1. L'utente digita una domanda in linguaggio naturale (NL) nella SearchBar del frontend;

2. Il frontend invia la domanda al backend via HTTP POST su `/api/query`;
3. Lo **SPARQL Agent** (GPT-4.1-mini) genera una query SPARQL a partire dalla domanda NL, utilizzando il contesto dell'ontologia (classi, proprietà, esempi);
4. Il **SPARQL Validator** controlla la correttezza sintattica della query generata; se invalida, l'agente ritenta (max 3 tentativi);
5. La query viene eseguita su **pyoxigraph** contro il file OWL;
6. Il **GraphBuilder** trasforma i risultati SPARQL in nodi e archi per il frontend (con metadati di tipo, label, colore);
7. Il frontend riceve i risultati e li visualizza nella ResultList e nel Knowledge Graph 2D interattivo.

2.4 Cache a Due Livelli

Per ottimizzare le performance, il sistema adotta una cache a due livelli:

- **Upstash Redis** (cloud, persistente, TTL 7 giorni): memorizza i risultati delle query SPARQL e le immagini (cover art) recuperate da Wikipedia. I risultati nulli non vengono cachati per consentire retry successivi;
- **Cache in-memory** (Python dict): mantiene in RAM label e tipi derivati dal grafo RDF per accessi rapidissimi durante la costruzione del grafo.

2.5 Tabella dei Componenti

La Tabella 2 elenca i principali componenti software con le rispettive responsabilità.

Tabella 2: Componenti principali e responsabilità

Componente	Responsabilità
<code>sparql_agent.py</code>	Traduzione NL → SPARQL via GPT-4.1-mini, retry
<code>sparql_validator.py</code>	Validazione sintattica SPARQL pre-esecuzione
<code>ontology_service.py</code>	Caricamento OWL, esecuzione query via pyoxigraph
<code>graph_builder.py</code>	Costruzione nodi/archi da risultati RDF per il frontend
<code>image_cache.py</code>	Recupero e cache immagini da Wikipedia API
<code>SearchBar.tsx</code>	Input NL, ricerca con autocomplete
<code>ResultList.tsx</code>	Visualizzazione risultati testuali con espansione dettaglio
<code>KnowledgeGraph.tsx</code>	Grafo 2D interattivo con force layout, zoom, filtri
<code>NodeDetail.tsx</code>	Pannello laterale con dettaglio nodo (proprietà, relazioni)
<code>NodeContextMenu.tsx</code>	Menu contestuale (click destro): espandi, rimuovi nodo
<code>SparqlViewer.tsx</code>	Visualizzazione query SPARQL generata (espandibile)

2.6 API Endpoint

La Tabella 3 riassume gli endpoint REST esposti dal backend.

Tabella 3: Endpoint API

Metodo	Endpoint	Descrizione
POST	<code>/api/query</code>	Ricerca NL → risultati + grafo
GET	<code>/api/node/{uri}</code>	Dettaglio nodo + sottografo
GET	<code>/api/image-search</code>	Cerca cover art da Wikipedia
GET	<code>/api/stats</code>	Statistiche ontologia
GET	<code>/api/health</code>	Health check

3 Panoramica dell'Ontologia

3.1 Dominio

L'ontologia modella il dominio dei **videogiochi** nel periodo 2010–2026, coprendo le seguenti entità principali:

- **VideoGame** — il prodotto videoludico con i suoi attributi (nome, data rilascio, punteggio Metacritic, paese d'origine);
- **Developer / Publisher** — gli studi di sviluppo e le case editrici;
- **Genre** — il genere (Action, RPG, Strategy, ...);
- **Platform** — la piattaforma di esecuzione (PS5, PC, Switch, ...);
- **Character** — i personaggi (giocanti e non);
- **Franchise** — le serie e i sotto-franchise;
- **Award** — i premi ricevuti;
- **GameEngine** — il motore grafico utilizzato;
- **GameMode** — la modalità di gioco (single-player, multiplayer, co-op, MMO);
- **Review / GameEvent** — recensioni ed eventi del ciclo di vita.

3.2 Namespace e Separazione Schema/Istanze

Il namespace dell'ontologia è:

`http://www.videogame-ontology.org/ontology#`

abbreviato in **vg**: in tutte le query SPARQL e nei riferimenti interni.

L'ontologia è strutturata in due file distinti che realizzano la classica separazione schema/istanze:

- **videogames.owl** — **schema**: contiene tutte le dichiarazioni di classi, proprietà, assiomi, disjointness e allineamenti esterni. Circa 945 triple;
- **videogames_pruned_2020.owl** — **insieme di istanze di demo**: contiene le istanze (giochi, developer, publisher, generi, piattaforme, ...) per il periodo 2020–2026. Circa 745.000 triple, 68.700 giochi.

3.3 Fonte Dati: Wikidata

Tutti i dati provengono da **Wikidata** [Foundation, 2024], il knowledge graph libero della Wikimedia Foundation. La popolazione avviene tramite query SPARQL al `https://query.wikidata.org/sparql`, mappando 13 proprietà Wikidata (identificatori P31, P136, P400, P577, P1712, ...) nelle corrispondenti proprietà dell'ontologia.

3.4 Pipeline di Popolamento

La pipeline completa si articola in quattro fasi:

1. **populate_wikidata.py** — esegue 204 iterazioni (17 anni $\times$ 12 mesi) di query SPARQL verso Wikidata, raccogliendo giochi, metadati e relazioni. Tempo stimato: ~ 5 ore con polite delay di 1 secondo tra le richieste.
2. **OWL-RL closure via owlrl** — applica la chiusura deduttiva OWL 2 RL per materializzare proprietà inverse, catene `subPropertyOf`, e inferenze da `someValuesFrom/hasValue/intersectionOf`. Da $\sim 1\text{M}$ a $\sim 1.4\text{M}$ triple.
3. **Pruning** — rimuove triple ridondanti o inutilizzate a runtime: `owl:sameAs` riflessivi, `gameDescription`, `officialWebsite`. Da $\sim 1.4\text{M}$ a $\sim 1.17\text{M}$ triple (completo) o $\sim 745\text{k}$ (demo 2020+).
4. **enrich_owl.py** — ripristina gli assiomi DL-only (FunctionalProperty, InverseFunctionalProperty, hasKey) dopo il reasoning owlrl.

3.5 Statistiche Riassuntive

La Tabella 4 riassume le principali metriche dello schema dell'ontologia.

Tabella 4: Statistiche dello schema dell'ontologia

Metrica	Valore
Classi totali	78
Classi primitive	61
Classi definite	17
Object Property	31
Datatype Property	26
Property chain axioms	3
Proprietà transitive	2
Coppie inverse	13
Blocchi AllDisjointClasses	12
Vincoli hasKey	3
Allineamenti equivalentClass	4 (Wikidata)
Allineamenti equivalentProperty	3 (Dublin Core)
Triple dello schema totali	~ 945

Lo schema è espresso in formato RDF/XML e può essere validato, arricchito e ragionato tramite gli script nella directory `ontology/` (cfr. Sezione 11 per la validazione e Sezione 9 per il reasoning).

4 Classi dello Schema dell'Ontologia

4.1 Gerarchia delle Classi

L'ontologia definisce **12 classi top-level**, ciascuna corrispondente a un'entità fondamentale del dominio, più un ricco albero di sottoclassi che raggiunge un totale di **78 classi** (di cui 61 primitive e 17 definite).

Le 12 classi top-level sono:

1. `vg:VideoGame` — il videogioco, classe centrale dell'ontologia;
2. `vg:Developer` — lo studio di sviluppo;
3. `vg:Publisher` — l'editore;
4. `vg:Genre` — il genere videoludico;
5. `vg:Platform` — la piattaforma hardware/software;
6. `vg:Character` — il personaggio (giocante o non);
7. `vg:Franchise` — la serie/franchise;
8. `vg:Award` — il premio;
9. `vg:GameEngine` — il motore grafico;
10. `vg:GameMode` — la modalità di gioco;
11. `vg:Review` — la recensione;
12. `vg:GameEvent` — l'evento del ciclo di vita.

4.2 Tabella Completa delle Classi Primitive

La Tabella 5 elenca tutte le 61 classi primitive (escluse le 17 definite, trattate nella Sezione 6), raggruppate per classe top-level di appartenenza.

Tabella 5: Classi primitive dell'ontologia

Classe	Parent	Descrizione
VideoGame (classe top-level)		
VideoGame	owl:Thing	Prodotto videoludico
ReleasedGame	VideoGame	Gioco ufficialmente rilasciato
EarlyAccessGame	VideoGame	Gioco in accesso anticipato
CancelledGame	VideoGame	Gioco cancellato prima del rilascio
DelistedGame	VideoGame	Gioco rimosso dagli store digitali
IndieGame	VideoGame	Gioco a budget ridotto, sviluppatore indipendente
AAAGame	VideoGame	Gioco ad alto budget, grande publisher
GOTYWinner	AwardWinningGame	Vincitore del premio Game of the Year
Developer (classe top-level)		
Developer	owl:Thing	Studio di sviluppo
FirstPartyDeveloper	Developer	Studio proprietario di un platform holder
ThirdPartyDeveloper	Developer	Studio indipendente
IndieDeveloper	Developer	Piccolo sviluppatore senza publisher
Publisher (classe top-level)		
Publisher	owl:Thing	Casa editrice
FirstPartyPublisher	Publisher	Divisione editoriale di un platform holder
ThirdPartyPublisher	Publisher	Casa editrice indipendente
Genre (classe top-level)		
Genre	owl:Thing	Genere videoludico
ActionGenre	Genre	Azione, sfide fisiche
RPGGenre	Genre	Gioco di ruolo, progressione
StrategyGenre	Genre	Strategia, pensiero tattico
PuzzleGenre	Genre	Rompicapo, problem-solving
SimulationGenre	Genre	Simulazione di sistemi reali/fittizi
SportsGenre	Genre	Sport, competizioni atletiche
RacingGenre	Genre	Corse, competizione su veicoli
AdventureGenre	Genre	Esplorazione, narrativa
HorrorGenre	Genre	Horror, paura, suspense
PlatformerGenre	Genre	Piattaforme, ostacoli
Platform (classe top-level)		
Platform	owl:Thing	Piattaforma di esecuzione
ConsolePlatform	Platform	Console dedicata
PlayStationPlatform	ConsolePlatform	Famiglia Sony PlayStation

Continua nella pagina successiva

Tabella 5 – continua dalla pagina precedente

Classe	Parent	Descrizione
XboxPlatform	ConsolePlatform	Famiglia Microsoft Xbox
NintendoPlatform	ConsolePlatform	Famiglia Nintendo
PCPlatform	Platform	Personal computer
WindowsPlatform	PCPlatform	Microsoft Windows
MacOSPlatform	PCPlatform	Apple macOS
LinuxPlatform	PCPlatform	Linux
MobilePlatform	Platform	Dispositivo mobile
IOSPlatform	MobilePlatform	Apple iOS
AndroidPlatform	MobilePlatform	Android
Character (classe top-level)		
Character	owl:Thing	Personaggio
PlayerCharacter	Character	Controllato dal giocatore
NonPlayerCharacter	Character	Controllato dall'IA
Franchise (classe top-level)		
Franchise	owl:Thing	Serie/franchise
SubFranchise	Franchise	Spin-off o sotto-serie
Award (classe top-level)		
Award	owl:Thing	Premio
IndustryAward	Award	Conferito da ente industriale
EditorialAward	Award	Conferito da testata editoriale
UserAward	Award	Determinato da voto popolare
GameEngine (classe top-level)		
GameEngine	owl:Thing	Motore grafico
ProprietaryEngine	GameEngine	Proprietario, non pubblico
OpenSourceEngine	GameEngine	Open-source
GameMode (classe top-level)		
GameMode	owl:Thing	Modalità di gioco
SinglePlayerMode	GameMode	Giocatore singolo
MultiplayerMode	GameMode	Multigiocatore
CoopMode	GameMode	Cooperativa
MMOMode	GameMode	Massively Multiplayer Online
GameEvent (classe top-level)		
GameEvent	owl:Thing	Evento del ciclo di vita
AnnouncementEvent	GameEvent	Annuncio pubblico
ReleaseEvent	GameEvent	Rilascio ufficiale
UpdateEvent	GameEvent	Aggiornamento/patch
DelistingEvent	GameEvent	Rimozione dagli store

4.3 Scelte di Modellazione

4.3.1 Partizione di Platform

La classe **Platform** è stata partizionata in tre rami principali (**ConsolePlatform**, **PCPlatform**, **MobilePlatform**), ciascuno ulteriormente specializzato:

Platform

```

+-- ConsolePlatform
|   +-- PlayStationPlatform
|   +-- XboxPlatform
|   +-- NintendoPlatform
+-- PCPlatform
|   +-- WindowsPlatform
|   +-- MacOSPlatform
|   +-- LinuxPlatform
+-- MobilePlatform
    +-- IOSPlatform
    +-- AndroidPlatform

```

Questa struttura consente interrogazioni a diversi livelli di granularità: <giochi disponibili su console> (`ConsolePlatform`), <giochi per PlayStation> (`PlayStationPlatform`), oppure <giochi per PC Windows> (`WindowsPlatform`). La partizione è resa semanticamente rigorosa da 5 blocchi `AllDisjointClasses` (cfr. Sezione 7). Ogni piattaforma Wikidata viene classificata al livello più specifico tramite le regole di reasoning (cfr. Sezione 9).

4.3.2 Generi come Classi Aperte

`Genre` ha 10 sottoclassi (`ActionGenre`, `RPGGenre`, ...) ma **non** viene dichiarata come esaustiva (`owl:oneOf`) né come coperta da un'unione. Questa scelta è deliberata: Wikidata può contenere generi non previsti dalla tassonomia attuale (es. <Battle Royale>, <Roguelike>). Un'ontologia chiusa impedirebbe l'aggiunta di nuovi generi senza modificare lo schema. Con la modellazione aperta, eventuali generi Wikidata non mappati restano istanze di `Genre` senza violare la semantica.

4.3.3 GameMode come Classe Top-Level Separata

`GameMode` è modellata come classe top-level separata (anziché come attributo di `VideoGame`) per consentire il pattern **Value Partition**: le sue 4 sottoclassi (`SinglePlayerMode`, `MultiplayerMode`, `CoopMode`, `MMOMode`) sono dichiarate mutuamente disgiunte, garantendo che un gioco con più modalità abbia asserzioni `hasGameMode` distinte e semanticamente non ambigue. Questa scelta permette inoltre di definire classi derivate (`SinglePlayerGame`, `MultiplayerGame`, `CoopGame`) tramite `owl:someValuesFrom` sulle modalità.

4.3.4 Review e GameEvent: Pattern N-ary Relation e Lifecycle Event

`Review` e `GameEvent` sono state introdotte come nuove classi top-level per modellare relazioni che richiedono reificazione:

- **Review** — reifica la relazione tra un gioco, un revisore e un punteggio. Senza la classe `Review`, sarebbe impossibile associare un punteggio (datatype property) a una specifica combinazione gioco-recensore. Segue il pattern **N-ary Relation** (cfr. Lez. 14);
- **GameEvent** — reifica eventi temporali come annunci, rilasci e aggiornamenti. Consente di associare date e metadati a eventi specifici del ciclo di vita di un gioco, seguendo il pattern **Lifecycle Event**.

5 Proprietà dello Schema dell'Ontologia

5.1 Object Property

La Tabella 6 elenca tutte le 31 object property definite nell'ontologia. Per semplicità, si omettono le proprietà inverse quando speculari; la colonna **Assiomi** indica le caratteristiche OWL 2 aggiuntive.

Tabella 6: Object Property

Proprietà	Domain	Range	Assiomi	Label
developedBy	VideoGame	Developer	Asym, Irr, inv developerOf	developed by
developerOf	Developer	VideoGame	Asym, Irr	developer of
publishedBy	VideoGame	Publisher	Asym, Irr, inv publisherOf	published by
publisherOf	Publisher	VideoGame	Asym, Irr	publisher of
hasGenre	VideoGame	Genre	Asym, Irr, inv genreOf	has genre
genreOf	Genre	VideoGame	Asym, Irr	genre of
availableOn	VideoGame	Platform	Irr, inv platformFor	available on
platformFor	Platform	VideoGame	Asym, Irr	platform for
hasCharacter	VideoGame	Character	Asym, Irr, inv appearsIn	has character
appearsIn	Character	VideoGame	Asym, Irr	appears in
belongsTo	VideoGame	Franchise	Asym, Irr, inv includes	belongs to
includes	Franchise	VideoGame	Asym, Irr	includes
wonAward	VideoGame	Award	Irr, inv awardWonBy	won award
awardWonBy	Award	VideoGame	Irr	award won by
sequelOf	VideoGame	VideoGame	Asym, Irr, Trans, inv hasSequel	sequel of
hasSequel	VideoGame	VideoGame	Asym, Irr	has sequel
hasGameMode	VideoGame	GameMode	Irr, inv modeInGame	has game mode
modeInGame	GameMode	VideoGame	Irr	mode in game
madeWith	VideoGame	GameEngine	Asym, Irr, inv engineUsedIn	made with
engineUsedIn	GameEngine	VideoGame	Asym, Irr	engine used in
partOfFranchise	Franchise	Franchise	Asym, Irr, Trans	part of franchise
subsidiaryOf	—	—	Asym, Irr, Trans	subsidiary of
sharedFranchiseWith	VideoGame	VideoGame	Sym, chain: belongsTo o includes	shared franchise with
sharesDeveloperWith	VideoGame	VideoGame	Sym, chain: developedBy o developerOf	shares developer with
sharesPublisherWith	VideoGame	VideoGame	Sym, chain: publishedBy o publisherOf	shares publisher with
reviewer	Review	—	—	reviewer
reviewedGame	Review	VideoGame	Funct, inv hasReview	reviewed game
hasReview	VideoGame	Review	—	has review
reviewOf	Review	VideoGame	—	review of
eventFor	GameEvent	VideoGame	inv hasEvent	event for
hasEvent	VideoGame	GameEvent	—	has event

Legenda: Asym = AsymmetricProperty, Irr = IrreflexiveProperty, Sym = SymmetricProperty, Trans = TransitiveProperty, Funct = FunctionalProperty, inv = inverseOf, chain = propertyChainAxiom.

5.2 Datatype Property

La Tabella 7 elenca tutte le 26 datatype property.

Tabella 7: Datatype Property

Proprietà	Domain	Range	subPropertyOf	Label
name	—	xsd:string	—	name
gameName	VideoGame	xsd:string	name	game name
gameDescription	VideoGame	xsd:string	—	game description
releaseDate	VideoGame	xsd:date	—	release date
metacriticScore	VideoGame	xsd:integer	—	Metacritic score
developerName	Developer	xsd:string	name	developer name
publisherName	Publisher	xsd:string	name	publisher name
genreName	Genre	xsd:string	name	genre name
platformName	Platform	xsd:string	name	platform name
characterName	Character	xsd:string	name	character name
franchiseName	Franchise	xsd:string	name	franchise name
awardName	Award	xsd:string	name	award name
awardYear	Award	xsd:integer	—	award year
description	—	xsd:string	—	description
officialWebsite	VideoGame	xsd:anyURI	—	official website
countryOfOrigin	VideoGame	xsd:string	—	country of origin
engineName	GameEngine	xsd:string	name	engine name
hasSteamAppId	VideoGame	xsd:string	—	Steam App ID
hasWikidataId	—	xsd:string	—	Wikidata ID
hasIGDBId	VideoGame	xsd:string	—	IGDB ID
reviewScore	Review	xsd:integer	—	review score
reviewDate	Review	xsd:date	—	review date
reviewerName	Review	xsd:string	—	reviewer name
eventDate	GameEvent	xsd:date	—	event date
budgetTier	VideoGame	xsd:string	—	budget tier
playerCount	VideoGame	xsd:integer	—	player count

5.3 Property Chain Axiom

L'ontologia definisce tre `owl:propertyChainAxiom`, che permettono di inferire relazioni transitive composte. In notazione DL (Description Logic):

1. `sharedFranchiseWith` $\equiv$ `belongsTo` $\circ$ `includes`
Due giochi condividono un franchise se il primo appartiene a un franchise e il franchise include il secondo.
2. `sharesDeveloperWith` $\equiv$ `developedBy` $\circ$ `developerOf`
Due giochi condividono lo sviluppatore se il primo è sviluppato da uno studio che ha sviluppato anche il secondo.
3. `sharesPublisherWith` $\equiv$ `publishedBy` $\circ$ `publisherOf`
Analogo per l'editore.

Queste catene sono materializzate dal reasoner OWL 2 RL (`owlrl`) e producono relazioni simmetriche tra giochi dello stesso franchise, studio o editore. Per evitare esplosioni combinatorie su franchise/studio/editore con migliaia di giochi, il materialiser mirato in `enrich_owl.py` limita il numero di coppie per gruppo tramite il flag `-max-group` (default 50).

5.4 Proprietà Transitive

Due proprietà sono dichiarate transitive:

- **sequelOf** — se A è sequel di B e B è sequel di C , allora A è sequel di C . Permette di tracciare catene di sequel (es. Dark Souls III $\rightarrow$ Dark Souls II $\rightarrow$ Dark Souls);
- **partOfFranchise** — per gerarchie di franchise con sotto-franchise. Es. $\langle \text{Mario Kart} \rangle \rightarrow \langle \text{Mario} \rangle$.

La transitività di **sequelOf** è particolarmente utile per query del tipo $\langle \text{tutti i sequel di Dark Souls} \rangle$ senza dover navigare manualmente la catena.

5.5 Coppie Inverse Sistematiche

L'ontologia definisce 13 coppie inverse, consentendo la navigazione bidirezionale del grafo. Per ogni proprietà “forward” esiste la corrispondente inversa:

- **developedBy** $\leftrightarrow$ **developerOf**
- **publishedBy** $\leftrightarrow$ **publisherOf**
- **hasGenre** $\leftrightarrow$ **genreOf**
- **availableOn** $\leftrightarrow$ **platformFor**
- **hasCharacter** $\leftrightarrow$ **appearsIn**
- **belongsTo** $\leftrightarrow$ **includes**
- **wonAward** $\leftrightarrow$ **awardWonBy**
- **sequelOf** $\leftrightarrow$ **hasSequel**
- **hasGameMode** $\leftrightarrow$ **modeInGame**
- **madeWith** $\leftrightarrow$ **engineUsedIn**
- **reviewedGame** $\leftrightarrow$ **hasReview**
- **eventFor** $\leftrightarrow$ **hasEvent**

La tredicesima coppia (**hasGenre** $\leftrightarrow$ **genreOf**) era già implicita nello schema originale. La dichiarazione esplicita di **owl:inverseOf** in una sola direzione è sufficiente: **owlrl** inferisce automaticamente la direzione opposta durante la materializzazione. Il validatore dello schema segnala 12 warning **INVERSE_NOT_BIDIRECTIONAL** proprio per questa asimmetria dichiarativa (cfr. Sezione 11).

6 Classi Definite

6.1 Panoramica

Le classi definite (*defined classes*) costituiscono uno degli aspetti più avanzati dell'ontologia. A differenza delle classi primitive, che richiedono l'asserzione esplicita **rdf:type**, le classi definite specificano condizioni necessarie e sufficienti (**owl:equivalentClass**) che permettono a un reasoner di **inferire automaticamente** l'appartenenza di un'istanza.

L'ontologia definisce **17 classi definite**, raggruppabili in sei categorie logiche. La Tabella 8 le elenca con la sintassi DL e il tipo di condizione.

Tabella 8: Classi definite

Classe	Tipo	Sintassi DL	Costrutti OWL 2
MultiplayerGame	S/N	$\text{VideoGame} \sqcap \exists \text{hasGameMode.MultiplayerMode}$	intersectionOf, some luesFrom
SinglePlayerGame	S/N	$\text{VideoGame} \sqcap \exists \text{hasGameMode.SinglePlayerMode}$	intersectionOf, some luesFrom
CoopGame	S/N	$\text{VideoGame} \sqcap \exists \text{hasGameMode.CoopMode}$	intersectionOf, some luesFrom
PCGame	S/N	$\text{VideoGame} \sqcap \exists \text{availableOn.PCPlatform}$	intersectionOf, some luesFrom

Tabella 8 – continua

Classe	Tipo	Sintassi DL	Costrutti OWL 2
ConsoleGame	S/N	VideoGame $\sqcap \exists \text{availableOn.ConsolePlatform}$	intersectionOf, some luesFrom
MobileGame	S/N	VideoGame $\sqcap \exists \text{availableOn.MobilePlatform}$	intersectionOf, some luesFrom
NintendoGame	S/N	VideoGame $\sqcap \exists \text{availableOn.NintendoPlatform}$	intersectionOf, some luesFrom
PlayStationGame	S/N	VideoGame $\sqcap \exists \text{availableOn.PlayStationPlatform}$	intersectionOf, some luesFrom
XboxGame	S/N	VideoGame $\sqcap \exists \text{availableOn.XboxPlatform}$	intersectionOf, some luesFrom
SequelGame	S/N	VideoGame $\sqcap \exists \text{sequelOf.VideoGame}$	intersectionOf, some luesFrom
StandaloneGame	S/N	VideoGame $\sqcap \neg \exists \text{sequelOf.VideoGame}$	intersectionOf, com mentOf
HighlyRatedGame	S/N	VideoGame $\sqcap \exists \text{metacriticScore} \geq 85$	intersectionOf, some luesFrom, datatype striction
RecentGame	S/N	VideoGame $\sqcap \exists \text{releaseDate} \geq 2020$	intersectionOf, some luesFrom, datatype striction
AwardWinningGame	S/N	VideoGame $\sqcap \exists \text{wonAward.Award}$	intersectionOf, some luesFrom
FranchiseGame	S/N	VideoGame $\sqcap \exists \text{belongsTo.Franchise}$	intersectionOf, some luesFrom
GameWithSingleDeveloper	N	VideoGame $\sqcap \leq 1 \text{ developedBy}$	subClassOf, maxCa nality
ExclusiveRelease	N	VideoGame $\sqcap \leq 1 \text{ availableOn}$	subClassOf, maxCa nality

Legenda: S/N = condizioni sufficienti e necessarie (equivalentClass); N = solo necessarie (subClassOf).

6.2 Analisi per Categoria

6.2.1 Mode-derived

Le classi `MultiplayerGame`, `SinglePlayerGame` e `CoopGame` classificano i giochi in base alla modalità di gioco. Un gioco viene inferito come `MultiplayerGame` se esiste almeno un'asserzione `hasGameMode` verso un'entità di tipo `MultiplayerMode`. Si noti che `SinglePlayerGame` e `MultiplayerGame` **non** sono dichiarate disgiunte: un gioco può supportare entrambe le modalità (es. *Call of Duty* ha sia campagna single-player che multiplayer).

Esempio: *Elden Ring* ha `hasGameMode SinglePlayerMode` e `hasGameMode MultiplayerMode` → inferito sia come `SinglePlayerGame` che come `MultiplayerGame`.

6.2.2 Platform-derived

Sei classi classificano i giochi per piattaforma a tre livelli di granularità: macrocategoria (`PCGame`, `ConsoleGame`, `MobileGame`) e famiglia specifica (`NintendoGame`, `PlayStationGame`, `XboxGame`). L'inferenza sfrutta la gerarchia `rdfs:subClassOf` delle piattaforme: se una piattaforma è di tipo `PlayStationPlatform`, il reasoner inferisce anche `ConsolePlatform` e `Platform`.

Esempio: God of War Ragnarök ha `availableOn` PS5 (tipo `PlayStationPlatform`) $\rightarrow$ inferito `PlayStationGame` e `ConsoleGame`.

6.2.3 Sequel-derived

`SequelGame` e `StandaloneGame` formano una partizione completa dei giochi basata sulla proprietà transitiva `sequelOf`:

- `SequelGame` $\equiv \exists \text{sequelOf}.\text{VideoGame}$: il gioco ha almeno un predecessore diretto;
- `StandaloneGame` $\equiv \neg \exists \text{sequelOf}.\text{VideoGame}$: il gioco non ha predecessori. Utilizza `owl:complementOf`, costruito non supportato da OWL 2 RL, e richiede pertanto il reasoner CONSTRUCT (cfr. Sezione 9).

Esempio: Elden Ring non ha `sequelOf` $\rightarrow$ `StandaloneGame`; Dark Souls III ha `sequelOf` Dark Souls II $\rightarrow$ `SequelGame`.

6.2.4 Quality-derived

`HighlyRatedGame` e `RecentGame` utilizzano **datatype restriction** con `xsd:minInclusive`, un costruito OWL 2 DL non coperto da OWL 2 RL (cfr. Lez. 17–18):

- `HighlyRatedGame` $\equiv \text{VideoGame} \sqcap \exists \text{metacriticScore}.\geq_{85}$ — giochi con Metacritic ≥ 85 ;
- `RecentGame` $\equiv \text{VideoGame} \sqcap \exists \text{releaseDate}.\geq_{2020-01-01}$ — giochi rilasciati dal 2020.

Esempio: Elden Ring (metacritic 96) $\rightarrow$ `HighlyRatedGame`.

6.2.5 Award e Franchise (preesistenti)

`AwardWinningGame` e `FranchiseGame` sono le due classi definite originali (V1), preservate nell'estensione V2:

- `AwardWinningGame`: giochi con almeno un premio (`wonAward`);
- `FranchiseGame`: giochi appartenenti a una serie (`belongsTo`).

6.2.6 Cardinality (solo necessarie)

`GameWithSingleDeveloper` e `ExclusiveRelease` sono le uniche due classi definite con **sole condizioni necessarie** (`rdfs:subClassOf` anziché `owl:equivalentClass`). Questa scelta è deliberata:

- `GameWithSingleDeveloper`: $\leq 1 \text{ developedBy}$. Condizione sufficiente sarebbe troppo forte (un gioco con esattamente 1 developer dovrebbe essere *solo* quello);
- `ExclusiveRelease`: $\leq 1 \text{ availableOn}$. Analogo ragionamento: un gioco esclusiva *ha al massimo* 1 piattaforma, ma non tutti i giochi con ≤ 1 piattaforma sono necessariamente esclusive.

Queste classi forniscono vincoli di integrità piuttosto che meccanismi di classificazione automatica.

6.3 Costrutti OWL 2 Utilizzati

La Tabella 9 riassume i costrutti OWL 2 impiegati nelle classi definite, con riferimento alle lezioni del corso.

Tabella 9: Costrutti OWL 2 nelle classi definite

Costrutto	Classi	Rif. Lezione
<code>owl:someValuesFrom</code>	MultiplayerGame, PCGame, SequelGame, ... (12)	Lez. 15 (esistenziale)
<code>owl:intersectionOf</code>	Tutte le 17 definite	Lez. 16 (intersezione)
<code>owl:complementOf</code>	StandaloneGame	Lez. 16 (complemento)
<code>owl:maxCardinality</code>	GameWithSingleDeveloper, ExclusiveRelease	Lez. 17–18
<code>owl:withRestrictions</code>	HighlyRatedGame, RecentGame	Lez. 17–18 (datatype)
<code>xsd:minInclusive</code>	HighlyRatedGame, RecentGame	Lez. 17–18

7 Disjointness

7.1 Blocchi AllDisjointClasses

L'ontologia dichiara **12 blocchi** `owl:AllDisjointClasses`, elencati nella Tabella 10. Ciascun blocco specifica un insieme di classi mutuamente disgiunte: nessuna istanza può appartenere a più di una classe nello stesso blocco.

Tabella 10: Blocchi AllDisjointClasses

#	Classi
1	VideoGame, Developer, Publisher, Genre, Platform, Character, Franchise, Award, GameEngine, GameMode, Review, GameEvent
2	ConsolePlatform, PCPlatform, MobilePlatform
3	PlayStationPlatform, XboxPlatform, NintendoPlatform
4	WindowsPlatform, MacOSPlatform, LinuxPlatform
5	IOSPlatform, AndroidPlatform
6	IndustryAward, EditorialAward, UserAward
7	PlayerCharacter, NonPlayerCharacter
8	ProprietaryEngine, OpenSourceEngine
9	ReleasedGame, EarlyAccessGame, CancelledGame, DelistedGame
10	IndieGame, AAAGame
11	SinglePlayerMode, MultiplayerMode, CoopMode, MMOMode
12	AnnouncementEvent, ReleaseEvent, UpdateEvent, DelistingEvent

7.2 Importanza delle Disjointness

Le dichiarazioni di disjointness svolgono due ruoli fondamentali:

1. **Rilevazione di inconsistenze nei dati Wikidata.** Se un'entità Wikidata venisse erroneamente classificata come, ad esempio, sia `ConsolePlatform` che `PCPlatform`, un reasoner DL completo (HermiT, Pellet) rileverebbe l'inconsistenza. Questo meccanismo funge da *sanity check* automatico sulla qualità dei dati;
2. **Ragionamento DL.** Le disjointness abilitano inferenze per contraddizione. Ad esempio, sapendo che `PlayerCharacter` e `NonPlayerCharacter` sono disgiunte, se un personaggio viene classificato come `PlayerCharacter`, si può inferire che **non** è `NonPlayerCharacter`.

7.3 Nota su SinglePlayerGame e MultiplayerGame

Si noti che `SinglePlayerGame` e `MultiplayerGame` **non** compaiono in alcun blocco di disjointness. La scelta è deliberata: nel dominio dei videogiochi, è comune che un gioco supporti sia la modalità

single-player che multiplayer (es. Grand Theft Auto V, Elden Ring, Call of Duty). Dichiararle disgiunte produrrebbe inconsistenze su una porzione significativa delle istanze.

Al contrario, le modalità `SinglePlayerMode`, `MultiplayerMode`, `CoopMode` e `MMOMode` (blocco #11) **sono** disgiunte: una specifica modalità di gioco è esattamente una di queste quattro.

8 Allineamento Esterno

8.1 `equivalentClass` verso Wikidata

L'ontologia dichiara 4 allineamenti `owl:equivalentClass` verso entità Wikidata, riportati nella Tabella 11.

Tabella 11: Allineamenti `equivalentClass` verso Wikidata

Classe VG	Entità Wikidata	Descrizione Wikidata
VideoGame	wd:Q7889	Video game
Developer	wd:Q210167	Video game developer
GameEngine	wd:Q193564	Game engine
Genre	wd:Q65956376	Video game genre

8.2 `equivalentProperty` verso Dublin Core

Tre datatype property sono allineate con il vocabolario Dublin Core [Initiative, 2020] tramite `owl:equivalentProperty` (Tabella 12).

Tabella 12: Allineamenti `equivalentProperty` verso Dublin Core

Proprietà VG	Proprietà DC	Label DC
releaseDate	dcterms:issued	Date Issued
gameName	dcterms:title	Title
gameDescription	dcterms:description	Description

8.3 Vantaggi dell'Allineamento

L'allineamento esterno, noto in letteratura come **ontology alignment** (cfr. Lez. 13), offre tre vantaggi principali:

1. **Interoperabilità.** Un'applicazione che consuma dati Dublin Core può automaticamente interpretare `vg:releaseDate` come sinonimo di `dcterms:issued`, senza bisogno di mapping manuali;
2. **Linkage automatico.** Le entità Wikidata esterne referenziate tra le istanze (es. `wd:Q12345`) sono automaticamente riconosciute come istanze di `vg:VideoGame` se Wikidata le classifica come `Q7889`;
3. **Riuso di standard consolidati.** Anziché reinventare vocabolari, l'ontologia si appoggia a standard ampiamente adottati (Dublin Core per metadati bibliografici, Wikidata per identificazione entità), seguendo il principio *Linked Data* di riutilizzare URI esistenti.

8.4 Limitazione Attuale

L'allineamento è dichiarato a livello dello schema ma non viene sfruttato dal reasoner OWL 2 RL, poiché `owl:rl` opera solo sul namespace locale. Per sfruttare appieno questi assiomi, sarebbe

necessario un reasoner DL completo (HermiT, Pellet) che possa importare anche le definizioni di classe da Wikidata.

9 Reasoning

9.1 Architettura a Due Livelli

Il sistema di reasoning adotta un'architettura a due livelli che combina un reasoner OWL 2 RL standard con un reasoner CONSTRUCT personalizzato per i costrutti non coperti dal profilo RL.

9.1.1 Livello 1: owlrl (OWL 2 RL)

owlrl [Chuc and contributors, 2024] implementa la chiusura deduttiva per il profilo OWL 2 RL [Motik et al., 2012], materializzando le triple inferite direttamente nel grafo RDF. I costrutti supportati includono:

- `rdfs:subClassOf` — propagazione del tipo verso l'alto;
- `owl:someValuesFrom` — inferenza da restrizioni esistenziali (es. se `G wonAward A`, allora `G rdfs:type AwardWinningGame`);
- `owl:hasValue` — inferenza da valore specifico;
- `owl:intersectionOf` — scomposizione di intersezioni;
- `owl:inverseOf` — materializzazione di proprietà inverse;
- `owl:SymmetricProperty` e `owl:TransitiveProperty`;
- `owl:propertyChainAxiom` — catene di proprietà.

Sull'insieme di istanze di test (3 giochi + 2 developer + piattaforme), owlrl produce un incremento da ~997 a ~2680 triple.

9.1.2 Livello 2: construct_reasoner.py (SPARQL CONSTRUCT)

21 regole CONSTRUCT colmano le lacune di OWL 2 RL, operando su costrutti che richiedono ragionamento al di fuori del profilo RL. La Tabella 13 elenca le regole.

Tabella 13: Regole CONSTRUCT del reasoner personalizzato

#	Descrizione	Cosa inferisce
R1	HighlyRatedGame da metacritic-Score ≥ 85	?g a vg:HighlyRatedGame
R2	RecentGame da releaseDate $\geq 2020-01-01$	?g a vg:RecentGame
R3	StandaloneGame: VideoGame senza sequelOf	?g a vg:StandaloneGame (negation-as-failure)
R4	SequelGame: VideoGame con sequelOf	?g a vg:SequelGame
R5a-f	Classificazione per piattaforma (PC, Console, Mobile, Nintendo, PS, Xbox)	?g a vg:PCGame, ecc.
R6a-c	Classificazione per modalità (Multiplayer, SinglePlayer, Coop)	?g a vg:MultiplayerGame, ecc.
R7	Inverse hasSequel: se A sequelOf B $\rightarrow$ B hasSequel A	?b vg:hasSequel ?a
R8	Inverse awardWonBy	?a vg:awardWonBy ?g

Tabella 13 – continua

#	Descrizione	Cosa inferisce
R9a–d	Inverse: <code>genreOf</code> , <code>platformFor</code> , <code>engineUsedIn</code> , <code>modeInGame</code>	completamento bidirezionale
R10	<code>sharedGenreWith</code> (pairwise)	<code>?g1 vg:sharedGenreWith ?g2</code>
R11	<code>sharedEngineWith</code> (pairwise)	<code>?g1 vg:sharedEngineWith ?g2</code>

9.2 Perché Alcuni Costrutti Non Sono in OWL 2 RL

Il profilo OWL 2 RL è deliberatamente limitato per garantire la trattabilità computazionale (polinomiale). Tre famiglie di costrutti esulano da questo profilo:

1. **Datatype restriction** (`HighlyRatedGame`, `RecentGame`). Le restrizioni su tipi di dato con `xsd:minInclusive` richiedono il confronto di valori literal, operazione non prevista dalle regole RL standard. Le regole R1 e R2 risolvono con un semplice filtro SPARQL (`FILTER(?s >= 85)`);
2. **complementOf puro** (`StandaloneGame`). OWL 2 RL supporta solo forme limitate di negazione. La regola R3 implementa la *negation-as-failure* con `FILTER NOT EXISTS`, assumendo la completezza dei dati (closed-world assumption sulla proprietà `sequelOf`);
3. **maxCardinality** (`GameWithSingleDeveloper`, `ExclusiveRelease`). Queste classi sono dichiarate con sole condizioni necessarie (`rdfs:subClassOf`) e non vengono materializzate in nuove triple `rdf:type`: fungono da vincoli di integrità verificabili da un reasoner DL completo.

9.3 Bug Critico: owl:sameAs Falsi e Soluzione

9.3.1 Il Problema

Durante lo sviluppo è emerso un bug critico: `owlrl` inferiva `owl:sameAs` falsi tra giochi distinti quando determinati assiomi `FunctionalProperty` / `InverseFunctionalProperty` / `hasKey` erano dichiarati su `DatatypeProperty`.

Meccanismo del bug: `owlrl` tratta i literal (es. stringhe, interi) come individui OWL a tutti gli effetti. Quando una `DatatypeProperty` è dichiarata `FunctionalProperty` (es. `gameName`), `owlrl` applica la regola RL standard: se due soggetti condividono lo stesso valore per una `FunctionalProperty`, allora sono `owl:sameAs`. Poiché molti giochi condividono lo stesso nome o lo stesso punteggio Metacritic, il reasoner li unificava erroneamente.

Esempio: tre giochi distinti (`GameA`, `GameB`, `GameC`) tutti con `wonAward` → tutti classificati come `AwardWinningGame` prima del fix; dopo il fix, solo quelli con effettivo `wonAward` vengono classificati (1 gioco su 3 nel test delle istanze).

9.3.2 La Soluzione

La soluzione adottata opera in tre fasi (implementata in `enrich_owl.py`):

1. **Sezione “DL-only axioms” separata.** Gli assiomi `FunctionalProperty` e `InverseFunctionalProperty` su `DatatypeProperty`, insieme agli `owl:hasKey`, sono dichiarati in una sezione dedicata del file OWL con un commento esplicativo (righe 1401–1484 di `videogames.owl`);
2. **Strip automatico prima del reasoning.** La funzione `strip_dl_only_axioms()` rimuove questi assiomi dal grafo in-memory prima di eseguire `owlrl`, prevenendo le inferenze errate;
3. **Restore dopo il reasoning.** La funzione `restore_dl_only_axioms()` ripristina gli assiomi nel file OWL serializzato finale, preservandone il valore dichiarativo per future applicazioni con reasoner DL completi.

Il flag `-keep-dl-axioms` permette di saltare lo strip per testing con reasoner DL.

9.3.3 Impatto

Dopo il fix: 0 owl:sameAs falsi su tutto l'insieme di istanze di test (2715 triple finali, nessun gioco unificato erroneamente). Le 12 famiglie di regole Python dirette in `enrich_owl.py` (`_materialise`) classificano correttamente tutti i giochi senza produrre falsi positivi.

9.4 Materialiser Mirato in `enrich_owl.py`

Oltre al reasoning owl + CONSTRUCT, `enrich_owl.py` implementa un materialiser Python diretto con 12 famiglie di regole che operano senza passare per SPARQL, risultando più veloce di `owlrl` full su insiemi di istanze di grandi dimensioni:

1. AwardWinningGame $\leftarrow$ game wonAward _;
2. FranchiseGame $\leftarrow$ game belongsTo _;
3. sharedFranchiseWith $\leftarrow$ belongsTo $\circ$ includes
4. sharesDeveloperWith $\leftarrow$ developedBy $\circ$ developerOf
5. sharesPublisherWith $\leftarrow$ publishedBy $\circ$ publisherOf
6. MultiplayerGame / SinglePlayerGame / CoopGame $\leftarrow$ hasGameMode
7. PCGame / ConsoleGame / MobileGame $\leftarrow$ availableOn
8. NintendoGame / PlayStationGame / XboxGame $\leftarrow$ availableOn
9. HighlyRatedGame $\leftarrow$ metacriticScore ≥ 85
10. RecentGame $\leftarrow$ releaseDate $\geq 2020-01-01$
11. SequelGame $\leftarrow$ game sequelOf _
12. StandaloneGame $\leftarrow$ VideoGame senza sequelOf

Le regole pairwise (3–5) sono protette dal cap `-max-group` (default 50) per evitare esplosioni combinatorie su franchise con migliaia di giochi.

10 Popolamento dell'Ontologia

10.1 Pipeline `populate_wikidata.py`

La popolazione delle istanze avviene tramite lo script `populate_wikidata.py`, che interroga l'endpoint SPARQL pubblico di Wikidata. La pipeline è strutturata in quattro fasi principali.

10.1.1 Query SPARQL verso Wikidata

Vengono eseguite 13 tipologie di query SPARQL, ciascuna mirata a estrarre una specifica categoria di dati:

1. **Core** — identificativi, nomi, date di rilascio (P31, P577);
2. **Generi** — mapping genere via P136;
3. **Piattaforme** — piattaforme via P400;
4. **Personaggi** — personaggi presenti nel gioco;
5. **Franchise** — serie di appartenenza via P179;
6. **Game Mode** — modalità di gioco via P404;
7. **Metacritic** — punteggio Metacritic via P1712;
8. **Engine** — motore grafico via P408;
9. **Paese** — paese d'origine via P495;
10. **Sito web** — sito ufficiale via P856;
11. **Premi** — premi ricevuti via P166;
12. **Descrizioni** — descrizioni testuali;
13. **Immagini** — URL copertina.

Per evitare i timeout dell'endpoint pubblico Wikidata (limite ~ 60 secondi), le query sono partizionate per `anno` $\times$ `mese`: 17 anni (2010–2026) $\times$ 12 mesi = **204 iterazioni**. Tra ogni

richiesta viene inserito un **polite delay** di 1 secondo. Il tempo totale di esecuzione è di circa **5 ore**.

10.1.2 Deduplicazione

Wikidata può restituire entità duplicate con URI diversi ma stesso nome normalizzato (es. varianti con suffissi come anno o piattaforma). La pipeline applica una normalizzazione del nome (lowercase, rimozione punteggiatura, trim whitespace) e rimuove **4.420 entità duplicate**, preservando la prima occorrenza.

10.1.3 OWL-RL Reasoning

Dopo la popolazione iniziale ($\sim 1.019.589$ triple grezze), viene applicata la chiusura deduttiva OWL 2 RL tramite `owlrl`:

- Materializzazione proprietà inverse: per ogni `developedBy`, viene aggiunto `developerOf`;
- Catene subPropertyOf: `gameName` $\rightarrow$ `name`;
- Inferenze da classi definite (`AwardWinningGame`, `FranchiseGame`).

Il grafo cresce da $\sim 1\text{M}$ a $\sim 1.4\text{M}$ triple.

10.1.4 Pruning

Lo step finale di pruning (`prune_owl.py`) rimuove triple non necessarie a runtime:

- `owl:sameAs` riflessivi (ogni entità Wikidata genera un `sameAs` con sé stessa): $\sim 253\text{k}$ triple rimosse;
- `gameDescription`: testi lunghi non usati nelle query;
- `officialWebsite`: URI non necessari per la ricerca semantica.

Risultato finale: $\sim 1.173.959$ triple (dataset completo 2010–2026).

10.2 Demo Subset 2020–2026

Per rispettare i limiti di RAM del deploy su Render (512 MB), la demo live utilizza un sottoinsieme dell'ontologia limitato ai giochi dal 2020 in poi (`prune_years.py`):

- ~ 745.400 triple;
- ~ 68.700 giochi;
- RAM a runtime: ~ 250 MB (pyoxigraph).

10.3 Performance: rdflib vs pyoxigraph

La Tabella 14 confronta le performance dei due store RDF utilizzati nel progetto. Il passaggio da `rdflib` (Python puro) a `pyoxigraph` (binding Rust) ha prodotto un miglioramento drastico.

Tabella 14: Confronto performance rdflib vs pyoxigraph

Metrica	rdflib	pyoxigraph (completo)	pyoxigraph (demo)
RAM a runtime	~ 1.550 MB	~ 389 MB	~ 250 MB
Tempo di caricamento	$\sim 41\text{s}$	$\sim 2.2\text{s}$	$\sim 1.5\text{s}$
Query SPARQL tipica	1–23s	0.08–0.37s	0.05–0.25s
Speedup query	—	5–62×	—

`pyoxigraph` [Tpt and contributors, 2024] garantisce un caricamento quasi istantaneo (< 3 secondi contro 41 secondi) e query 5–62 volte più veloci, rendendo l'esperienza utente fluida anche su hardware limitato.

11 Validazione dello Schema

11.1 validate_tbox.py

Lo script `validate_tbox.py` implementa un validatore statico dello schema che esegue **10 controlli di consistenza** sullo schema dell'ontologia. La Tabella 15 elenca i controlli con codice e severità.

Tabella 15: Controlli di validazione dello schema

#	Codice	Descrizione
1	DANGLING_REFERENCE	Riferimenti a entità non dichiarate (ERROR)
2	UNSATISFIABLE_DISJOINT	Classi con superclassi disgiunte (ERROR)
3	CIRCULAR_SUBCLASS	Cicli nella gerarchia <code>rdfs:subClassOf</code> (ERROR)
4	INVERSE_CONSISTENCY	Consistenza <code>owl:inverseOf</code> bidirezionale (WARNING)
5	PROPERTY_TYPE_CONFLICT	Tipi di proprietà mutuamente esclusivi (ERROR)
6	MISSING_DOMAIN_RANGE	Domain/range mancanti su proprietà (WARNING)
7	MISSING_LABEL	Label <code>rdfs:label</code> mancanti (WARNING)
8	UNUSED_CLASSES	Classi dichiarate mai referenziate (WARNING)
9	RESTRICTION_COMPLETENESS	Restrizioni incomplete (ERROR/WARNING)
10	DATATYPE_REST_SYNTAX	Sintassi datatype restriction (WARNING)

11.2 Risultato della Validazione

La validazione sullo schema attuale (`videogames.owl`) produce il seguente risultato:

- **0 ERROR** — nessuna violazione critica;
- **18 WARNING** — tutti benigni e giustificati;
- **Esito:** PASS.

11.3 Analisi dei 18 Warning

I 18 warning si dividono in due categorie:

11.3.1 12 Warning INVERSE_NOT_BIDIRECTIONAL

Il validatore segnala che, per 12 coppie inverse, solo una direzione dichiarata esplicitamente `owl:inverseOf` (es. `developedBy owl:inverseOf developerOf` ma non viceversa). Questo è intenzionale:

- `owl:rl` inferisce automaticamente la direzione opposta durante la materializzazione, quindi la dichiarazione bidirezionale sarebbe ridondante;
- Lo schema originale V1 dichiarava solo la direzione “forward”; la V2 ha aggiunto le inverse come nuove proprietà, ma l'`inverseOf` inverso non è stato aggiunto per non duplicare assiomi già inferibili.

11.3.2 6 Warning MISSING_DOMAIN / MISSING_RANGE

Riguardano sei proprietà senza dominio o range dichiarato:

- `name`, `description` — super-proprietà generiche intenzionalmente polimorfiche (usate come base per `gameName`, `developerName`, ecc. tramite `rdfs:subPropertyOf`), quindi senza dominio fisso;
- `hasWikidataId` — proprietà trasversale applicabile a qualsiasi risorsa con un identificativo Wikidata, intenzionalmente senza dominio;

- **subsidiaryOf** — senza dominio/range per consentire l’uso sia su Developer che su Publisher;
- **reviewer** — range non specificato perché il revisore può essere un’entità testuale o un URI esterno.

Tutti questi warning sono giustificati da scelte di design e non costituiscono problemi di qualità dell’ontologia.

12 Frontend e Backend

12.1 Frontend: React + TypeScript + Vite

L’interfaccia utente è un’applicazione single-page costruita con **React 18**, **TypeScript**, **Vite** e **Tailwind CSS** [Meta Platforms, 2024]. Il componente centrale è il **Knowledge Graph 2D**, realizzato con la libreria **react-force-graph-2d**, che visualizza i risultati delle query come un grafo interattivo physics-based.

12.1.1 Componenti Principali

I sei componenti React che compongono l’interfaccia sono:

- **SearchBar** — input di ricerca con autocomplete. L’utente digita domande in linguaggio naturale e riceve suggerimenti;
- **ResultList** — elenca i risultati testuali della query con metadati (nome, anno, punteggio). Ogni risultato è cliccabile per espandere il dettaglio;
- **KnowledgeGraph** — il grafo interattivo 2D con force-directed layout. Supporta zoom, pan, hover (evidenzia le connessioni dirette), e un pulsante “recentra” per zoom-to-fit;
- **NodeDetail** — pannello laterale che mostra tutte le proprietà e relazioni del nodo selezionato. Include link cliccabili per espandere altri nodi nel grafo;
- **NodeContextMenu** — menu contestuale al click destro su un nodo: opzioni per espandere il nodo (aggiungere i vicini al grafo) o rimuoverlo;
- **SparqlViewer** — pannello espandibile che mostra la query SPARQL generata dall’agente, utile per debugging e trasparenza.

12.1.2 Caratteristiche del Grafo Interattivo

Il Knowledge Graph offre un’esperienza di navigazione ricca:

- **Nodi colorati per tipo:** VideoGame (blu), Developer (verde), Publisher (arancione), Genre (viola), Platform (rosso), ecc.;
- **VideoGame come rettangoli portrait:** i nodi gioco sono visualizzati come riquadri verticali con immagine di copertina (da Wikipedia) o emoji placeholder, differenziandosi dagli altri nodi circolari;
- **Animazione physics-based:** forza di attrazione/repulsione tra i nodi con smorzamento progressivo;
- **Long-press su mobile:** supporto touch per dispositivi mobili;
- **Filtro per tipo:** legenda interattiva che permette di nascondere/mostrare intere categorie di nodi con un click;
- **Hover:** evidenzia le connessioni dirette del nodo, portandolo in primo piano (z-order).

12.2 Backend: FastAPI Async

Il backend è sviluppato con **FastAPI** [Ramírez and contributors, 2024] in modalità asincrona, esponendo 6 endpoint REST (cfr. Tabella 3, Sezione 2). I componenti chiave sono:

12.2.1 SPARQL Agent

Il cuore intelligente del sistema è lo `sparql_agent.py`, che utilizza **GPT-4.1-mini** di OpenAI [OpenAI, 2025] per tradurre domande in linguaggio naturale in query SPARQL. Il prompt di sistema include:

- Il contesto completo dell'ontologia (classi, proprietà, esempi);
- I prefissi SPARQL (`vg:`, `rdf:`, `rdfs:`, `xsd:`);
- Esempi di domande e query corrispondenti (*few-shot prompting*);
- Istruzioni per generare solo la query SPARQL, senza testo aggiuntivo.

Il flusso include un meccanismo di **retry automatico** (max 3 tentativi):

1. L'agente genera una query SPARQL;
2. Il `sparql_validator` ne verifica la correttezza sintattica;
3. Se la query è sintatticamente invalida o produce zero risultati, l'agente riceve il messaggio di errore e ritenta con un prompt correttivo.

12.2.2 GraphBuilder

Il `graph_builder.py` trasforma i risultati SPARQL (binding di variabili) in una struttura JSON adatta al frontend:

- **Nodi**: URI, label, tipo (classe OWL), colore, forma, immagine;
- **Archi**: soggetto, predicato, oggetto, label della relazione.

Il builder arricchisce i nodi con metadati derivati dal grafo RDF (tipo tramite `rdf:type`, label tramite le proprietà `*Name`).

12.2.3 Cache a Due Livelli

- **Upstash Redis** [Inc., 2025]: cache cloud persistente con TTL di 7 giorni per risultati query e URL immagini;
- **Cache in-memory**: dizionario Python per label e tipi, popolato al caricamento dell'ontologia e consultabile in $O(1)$.

12.3 Endpoint API

Si rimanda alla Tabella 3 nella Sezione 2 per l'elenco completo degli endpoint REST.

13 Conclusione

13.1 Riepilogo dei Risultati

Il progetto **Videogame Semantic Search** ha prodotto un sistema completo di ricerca semantica che integra:

- **Un'ontologia OWL 2** espressiva e ben strutturata, evoluta da 11 a 78 classi (61 primitive + 17 definite), con 31 object property, 26 datatype property, 13 coppie inverse, 3 property chain, 2 proprietà transitive, 12 blocchi di disjointness e allineamento esterno a Wikidata e Dublin Core;
- **Un sistema di reasoning a due livelli** (OWL 2 RL + 21 regole CONSTRUCT) funzionante end-to-end, con 0 falsi `owl:sameAs` dopo il fix del bug critico degli assiomi DL-only;
- **Un knowledge graph popolato** da Wikidata con oltre 1.1 milioni di triple (dataset completo 2010–2026) e ~745k triple nel sottoinsieme demo (2020–2026, ~68.700 giochi);
- **Un'interfaccia web interattiva** con ricerca in linguaggio naturale, agente SPARQL intelligente, grafo 2D navigabile e filtri per tipo;

- **Performance ottimizzate** grazie al passaggio da rdflib a pyoxigraph (5–62× speedup sulle query, caricamento in <3s, RAM ~250 MB).

13.2 Possibili Estensioni Future

Il sistema offre numerose direzioni di sviluppo futuro:

1. **Reasoner DL completo** — integrare HermiT o Pellet tramite `owlready2` per ragionare sui costrutti DL-only (datatype restriction, complementOf, maxCardinality) senza bisogno delle regole CONSTRUCT compensate. Questo abiliterebbe anche la rilevazione automatica di inconsistenze tra le istanze;
2. **Validazione delle istanze con SHACL** — definire SHACL shapes per validare la conformità delle istanze (es. ogni VideoGame deve avere almeno un `gameName`, le date devono essere nel formato corretto);
3. **SWRL rules reali** — sostituire le 21 regole CONSTRUCT con autentiche SWRL rules, sfruttando un reasoner che le supporti nativamente;
4. **Estensione della pipeline Wikidata** — aggiungere query per recuperare `hasSteamAppId`, `budgetTier`, `playerCount` e la relazione `sequelOf`, attualmente non popolate dai dati Wikidata;
5. **Internazionalizzazione del frontend** — supporto multilingua per l'interfaccia e le query in linguaggio naturale (inglese, francese, ...);
6. **Visualizzazione avanzata delle classi definite** — nel grafo interattivo, evidenziare con badge o colori speciali i nodi che sono stati classificati automaticamente dal reasoner (es. bordino dorato per `HighlyRatedGame`, icona controller per `MultiplayerGame`);
7. **SPARQL endpoint pubblico** — esporre un endpoint SPARQL standard (RESTful o tramite Triplestore) per consentire query federate da applicazioni esterne.

Il progetto dimostra come le tecnologie del Semantic Web — ontologie formali, reasoning automatico, knowledge graph e query SPARQL — possano essere integrate in un'applicazione web moderna e performante, offrendo un'esperienza di ricerca semantica che va ben oltre la semplice ricerca per parole chiave.

A Esempio di Istanze di Test

Per la verifica end-to-end del sistema di reasoning, è stato utilizzato un insieme di istanze di test minimale (`test_abox.ttl`) contenente 3 giochi, 2 developer e alcune piattaforme. Il seguente estratto in Turtle illustra la struttura:

```
@prefix vg: <http://www.videogame-ontology.org/ontology#> .

vg:GameA a vg:VideoGame ;
  vg:gameName "Elden Ring" ;
  vg:releaseDate "2022-02-25"^^xsd:date ;
  vg:metacriticScore 96 ;
  vg:developedBy vg:Dev1 ;
  vg:hasGenre vg:RPGGenreInstance ;
  vg:hasGameMode vg:SinglePlayerMode , vg:MultiplayerMode ;
  vg:availableOn vg:PS5 , vg:PC_Windows ;
  vg:wonAward vg:TGAAward2022 .

vg:GameB a vg:VideoGame ;
  vg:gameName "Hollow Knight" ;
  vg:releaseDate "2017-02-24"^^xsd:date ;
  vg:metacriticScore 87 ;
  vg:developedBy vg:Dev2 ;
  vg:availableOn vg:NintendoSwitch ;
```

```

    vg:hasGameMode vg:SinglePlayerMode .

vg:GameC a vg:VideoGame ;
  vg:gameName "Dark Souls III" ;
  vg:sequelOf vg:GameX ;
  vg:developedBy vg:Dev1 .

```

Listing 1: Estratto del test delle istanze (Turtle)

A.1 Risultati della Verifica

Dopo il reasoning completo (owlrl + CONSTRUCT), il grafo di test produce:

- **2.715 triple finali** (dalle ~997 iniziali);
- **0 owl:sameAs falsi** — il fix degli assiomi DL-only previene l'unificazione errata dei giochi;
- **Classificazioni corrette:**
 - **GameA** → MultiplayerGame, SinglePlayerGame, ConsoleGame, PlayStationGame, PCGame, HighlyRatedGame, RecentGame, AwardWinningGame, StandaloneGame;
 - **GameB** → SinglePlayerGame, ConsoleGame, NintendoGame, HighlyRatedGame, StandaloneGame (non RecentGame: 2017);
 - **GameC** → SequelGame, ConsoleGame (non SinglePlayerGame/MultiplayerGame: nessun game mode asserito).

B Sintassi DL delle Classi Definite

Di seguito si riporta la sintassi DL compatta di tutte le 17 classi definite dell'ontologia. La notazione $\sqcap$ indica intersezione (`owl:intersectionOf`), $\exists$ indica restrizione esistenziale (`owl:someValuesFrom`), $\neg$ indica complemento (`owl:complementOf`), $\leq$ indica cardinalità massima (`owl:maxCardinality`).

$$\begin{aligned}
 \text{MultiplayerGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{MultiplayerMode} \\
 \text{SinglePlayerGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{SinglePlayerMode} \\
 \text{CoopGame} &\equiv \text{VideoGame} \sqcap \exists \text{hasGameMode}.\text{CoopMode} \\
 \text{PCGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{PCPlatform} \\
 \text{ConsoleGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{ConsolePlatform} \\
 \text{MobileGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{MobilePlatform} \\
 \text{NintendoGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{NintendoPlatform} \\
 \text{PlayStationGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{PlayStationPlatform} \\
 \text{XboxGame} &\equiv \text{VideoGame} \sqcap \exists \text{availableOn}.\text{XboxPlatform} \\
 \text{SequelGame} &\equiv \text{VideoGame} \sqcap \exists \text{sequelOf}.\text{VideoGame} \\
 \text{StandaloneGame} &\equiv \text{VideoGame} \sqcap \neg \exists \text{sequelOf}.\text{VideoGame} \\
 \text{HighlyRatedGame} &\equiv \text{VideoGame} \sqcap \exists \text{metacriticScore}.\geq_{85} \\
 \text{RecentGame} &\equiv \text{VideoGame} \sqcap \exists \text{releaseDate}.\geq_{2020-01-01} \\
 \text{AwardWinningGame} &\equiv \text{VideoGame} \sqcap \exists \text{wonAward}.\text{Award} \\
 \text{FranchiseGame} &\equiv \text{VideoGame} \sqcap \exists \text{belongsTo}.\text{Franchise} \\
 \text{GameWithSingleDeveloper} &\sqsubseteq \text{VideoGame} \sqcap \leq_1 \text{developedBy} \\
 \text{ExclusiveRelease} &\sqsubseteq \text{VideoGame} \sqcap \leq_1 \text{availableOn}
 \end{aligned}$$

Le prime 15 classi utilizzano $\equiv$ (condizioni necessarie e sufficienti, `owl:equivalentClass`); le ultime due usano $\sqsubseteq$ (sole condizioni necessarie, `rdfs:subClassOf`).

Bibliografia

- Edmond Chuc and contributors. owlrl: OWL-RL reasoning for RDFLib, 2024. URL <https://github.com/RDFLib/owlrl>.
- Wikimedia Foundation. Wikidata: The Free Knowledge Base, 2024. URL <https://www.wikidata.org/>.
- Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993. doi: 10.1006/knac.1993.1008.
- Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. OWL 2 Web Ontology Language Primer (Second Edition). Technical report, W3C, 2012. URL <https://www.w3.org/TR/owl2-primer/>.
- Upstash Inc. Upstash Redis, 2025. URL <https://upstash.com/>.
- Dublin Core Metadata Initiative. DCMI Metadata Terms. Technical report, DCMI, 2020. URL <https://www.dublincore.org/specifications/dublin-core/dcmi-terms/>.
- Inc. Meta Platforms. React: A JavaScript library for building user interfaces, 2024. URL <https://react.dev/>.
- Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. OWL 2 Web Ontology Language Profiles (Second Edition). Technical report, W3C, 2012. URL <https://www.w3.org/TR/owl2-profiles/>.
- OpenAI. GPT-4.1 mini: OpenAI API, 2025. URL <https://platform.openai.com/>.
- Sebastián Ramírez and contributors. FastAPI: modern web framework for building APIs, 2024. URL <https://fastapi.tiangolo.com/>.
- Tpt and contributors. pyoxigraph: Python bindings for Oxigraph, 2024. URL <https://github.com/oxigraph/oxigraph>.